

ENZO ESPIÑO CORRAL

Tutor: Andrea Chinnici

IES El Lago

CAMPUS LAGO

Documentación Técnica



TFC DAW

2026

Índice

1. Descripción del proyecto
2. Stack tecnológico
3. Arquitectura del sistema
4. Estructura de ficheros
5. Cómo levantar el proyecto
6. Variables de entorno
7. Base de datos
8. API REST
9. Frontend
10. Autenticación y seguridad
11. Funcionalidades principales
12. Panel de administración
13. Recuperación de contraseña

1. Descripción del proyecto

Campus Lago es un proyecto de una red social privada para IES El Lago, desarrollada como Trabajo de Fin de Ciclo del grado de Desarrollo de Aplicaciones Web. La idea es ofrecer al centro un espacio digital propio e independiente donde alumnos y profesores puedan publicar contenido, comentar, enviarse mensajes directos, seguirse entre sí y recibir notificaciones, sin depender de plataformas externas como Instagram o Twitter.

El acceso está restringido a cuentas con dominio @educa.madrid.org. Cualquier persona con esa dirección puede solicitar el registro, pero no accede directamente: su solicitud queda pendiente hasta que el administrador la aprueba y le asigna un rol (alumno o profesor). Esto garantiza que solo acceden personas pertenecientes al centro.

2. Stack tecnológico

Frontend: HTML5, CSS3 y JavaScript

Backend: Node.js 18 con Express 4

Base de datos: MySQL 8

Autenticación: JWT y bcryptjs

Almacenamiento de imágenes: Cloudinary

Contenedores: Docker

Control de versiones: Git y GitHub

Despliegue: Netlify para el frontend, Render para el backend y Clever Cloud para la base de datos

3. Arquitectura del sistema

El sistema está dividido en tres capas separadas. El frontend es una aplicación web estática: ficheros HTML, CSS y JS que el navegador descarga y ejecuta directamente. El backend es una API REST bajo el prefijo /api. La base de datos es MySQL, que tiene 11 tablas relacionales.

Además se usa Cloudinary como servicio externo para almacenar y servir las imágenes que suben los usuarios.

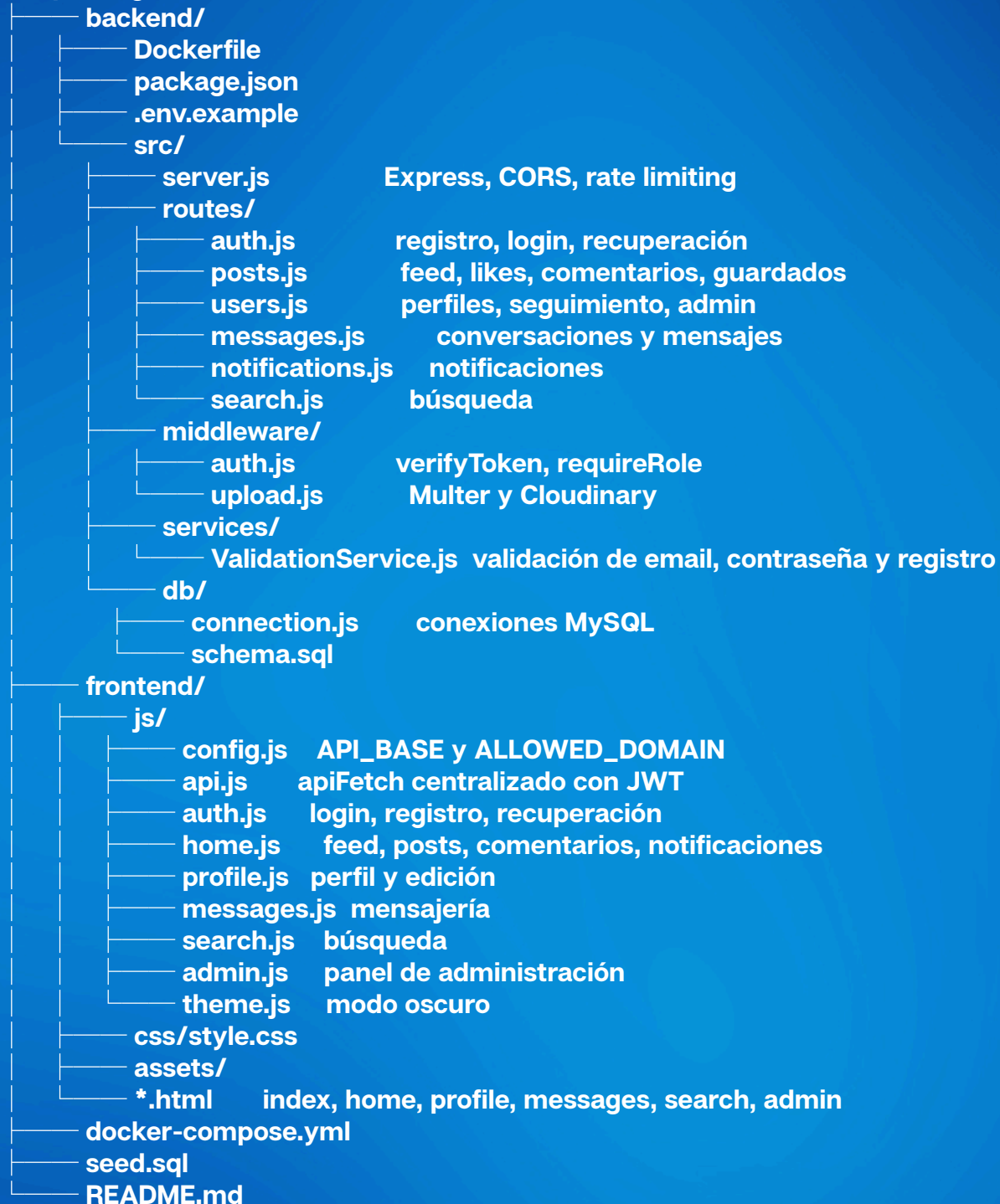
La comunicación entre frontend y API se hace por HTTPS con JSON. El fichero frontend/js/config.js detecta de forma automática si se está trabajando en local o desde producción:

```
const API_BASE = isLocal ? 'http://localhost:3000/api' : 'https://campus-lago.onrender.com/api';
```

Para la autenticación, al hacer login la API devuelve un JWT que el frontend guarda en localStorage y envía en cada petición con la cabecera Authorization: Bearer <token>.

4. Estructura de ficheros

campus-lago/



5. Cómo levantar el proyecto

Local

Solo hace falta tener Docker Desktop instalado y en marcha.

```
git clone https://github.com/9daemons/tfc-campuslago.git
cd tfc-campuslago
```

Crear el fichero backend/.env.docker con las variables necesarias (ver sección 6) y después ejecutar:

```
docker-compose up --build
```

La primera vez tarda unos 2 minutos en descargar imágenes y construir el backend.

Frontend disponible en <http://localhost:5500>

API disponible en <http://localhost:3000/api>

Para reiniciar la base de datos desde cero:

```
docker-compose down -v && docker-compose up --build
```

Credenciales de prueba (entorno local)

Contraseña para todos: password

admin@educa.madrid.org Administrador

ana.lopez@educa.madrid.org Profesor

laura.garcia@educa.madrid.org Alumno

Producción

Frontend en Netlify: directorio de publicación frontend/, sin comando de build. Se redespiega automáticamente con cada push a main.

Backend en Render: directorio raíz backend/, build con `npm ci --omit=dev`, inicio con `node src/server.js`. Las variables de entorno se configuran en el panel de Render, sin subir el fichero .env al repositorio. La URL de producción es <https://campuslago.onrender.com>.

Base de datos en Clever Cloud: addon MySQL gestionado. Las credenciales se añaden como variables de entorno en Render. El esquema se inicializa ejecutando `schema.sql` desde phpMyAdmin.

Cada push a main redespiega automáticamente tanto Netlify como Render.

6. Variables de entorno

PORT	puerto del servidor (3000)
FRONTEND_URL	URL del frontend para CORS
DB_HOST	host de la base de datos MySQL
DB_PORT	puerto MySQL (3306)
DB_NAME	nombre de la base de datos
DB_USER	usuario de MySQL
DB_PASSWORD	contraseña de MySQL
JWT_SECRET	clave secreta para firmar los tokens
JWT_EXPIRES_IN	duración del token (7d)
CLOUDINARY_CLOUD_NAME	nombre del cloud en Cloudinary
CLOUDINARY_API_KEY	clave de API de Cloudinary
CLOUDINARY_API_SECRET	secreto de API de Cloudinary
MAX_FILE_SIZE	tamaño máximo de imagen en bytes

7. Base de datos

El esquema completo está en backend/src/db/schema.sql y define 11 tablas:

users: usuarios del sistema.

Campos principales: id, email, username, password_hash, full_name, role (alumno, profesor o admin), bio, avatar, is_active, last_login y created_at.

El campo last_login se actualiza en cada inicio de sesión y se usa para el indicador de actividad reciente en los perfiles.

registration_requests: solicitudes de registro pendientes. Cuando alguien se registra no se crea el usuario directamente, sino una fila aquí con status = pending que el admin aprueba o rechaza.

password_resets: pins de recuperación de contraseña. Cada pin tiene expiración de 15 minutos y un campo used para marcarlo tras usarse, de modo que no pueda reutilizarse.

posts: publicaciones. Incluye post_type (oficial o alumno).

likes, comments, saved_posts: interacciones con publicaciones. Los registros de likes y saved_posts usan clave primaria compuesta (user_id, post_id) para evitar duplicados. Los comentarios incluyen user_id para controlar quién puede borrarlos.

follows: seguimiento entre usuarios, con clave primaria compuesta (follower_id, following_id).

conversations, conversation_members, messages: sistema de mensajería. Una conversación puede ser directa (is_group = false) o grupal (is_group = true, con nombre). conversation_members relaciona usuarios con conversaciones.

notifications: like, comentario, follow, mensaje y sistema.

Todas las claves foráneas usan ON DELETE CASCADE, así que borrar un usuario limpia automáticamente todos sus datos.

8. API REST

Todas las rutas están bajo /api. Las de autenticación son públicas; el resto requieren JWT válido en la cabecera Authorization.

Autenticación

POST /api/auth/register solicita el registro (queda pendiente)
POST /api/auth/login inicia sesión y devuelve el JWT
POST /api/auth/forgot-password pide un PIN de recuperación
POST /api/auth/reset-password cambia la contraseña con el PIN
GET /api/auth/me datos del usuario en sesión

Publicaciones

GET /api/posts/feed/official publicaciones oficiales del centro
GET /api/posts/feed/student publicaciones de alumnos y profesores
POST /api/posts nueva publicación (texto, imagen o ambas)
DELETE /api/posts/:id borra una publicación
POST /api/posts/:id/like alterna like o dislike
POST /api/posts/:id/save guarda o deja de guardar
GET /api/posts/:id/comments devuelve los comentarios
POST /api/posts/:id/comments añade un comentario
DELETE /api/posts/:id/comments/:commentId borra un comentario

Usuarios

GET /api/users/:username perfil de un usuario
GET /api/users/:username/posts publicaciones del usuario
PUT /api/users/me/profile actualiza nombre, bio y avatar
POST /api/users/:id/follow sigue o deja de seguir
GET /api/users/me/suggested hasta 8 usuarios sugeridos
Administración (solo rol admin)
GET /api/users/admin/requests solicitudes de registro pendientes
POST /api/users/admin/requests/:id/approve aprueba la solicitud y crea el usuario
POST /api/users/admin/requests/:id/reject rechaza la solicitud
GET /api/users/admin/users lista todos los usuarios
PUT /api/users/admin/users/:id/role cambia el rol del usuario
PUT /api/users/admin/users/:id/suspend suspende o reactiva la cuenta
POST /api/users/admin/users/:id/reset-pin genera un PIN de recuperación
DELETE /api/users/admin/users/:id elimina la cuenta
GET /api/users/admin/stats estadísticas globales

Mensajería

GET /api/messages/conversations lista de conversaciones
POST /api/messages/conversations crea una conversación
GET /api/messages/conversations/:id mensajes de una conversación
POST /api/messages/conversations/:id/messages envía un mensaje

Notificaciones, búsqueda y estado

GET /api/notifications lista de notificaciones
PUT /api/notifications/read-all marca todas como leídas
GET /api/search?q=... búsqueda global

9. Frontend

Cada sección tiene su propio fichero HTML. No hay uso de frameworks. La navegación usa enlaces normales entre páginas.

index.html: login, registro y recuperación de contraseña por PIN.

home.html: feed con posts mezclados por fecha. El sidebar izquierdo muestra mensajes recientes o, si el usuario es admin, las solicitudes de registro pendientes. El derecho muestra sugeridos o estadísticas según el rol.

profile.html: avatar con indicador de actividad (verde si estuvo activo en la última hora, naranja si fue en las últimas 24), estadísticas, bio, rol y publicaciones del usuario.

messages.html: lista de conversaciones y chat con refresco automático cada 3 segundos.

search.html: búsqueda en tiempo real (con un ligero retraso).

admin.html: panel de administración, visible solo para el rol admin.

Todo lo común (peticiones, token, escapeHtml, timeAgo, avatarUrl) está en api.js y se carga en todas las páginas. El modo oscuro se gestiona con la clase html.dark. Para evitar un parpadeo blanco al navegar theme.js se carga en el head.

10. Autenticación y seguridad

JWT: al hacer login el servidor genera un token con JWT_SECRET que incluye id, email, username y role. Dura 7 días. Si la API devuelve 401, api.js redirige al login de forma automática.

Contraseñas: se guardan con bcryptjs. Para registrarse se exige mínimo 8 caracteres, al menos una mayúscula y un número.

Restricción de dominio: el email debe ser @educa.madrid.org. Se valida tanto en el frontend en config.js como en el backend en ValidationService.

Rate limiting: las rutas de autenticación admiten máximo 20 peticiones por IP cada 15 minutos. Express está configurado con trust proxy para que el límite funcione correctamente detrás del proxy de Render.

CORS: el servidor solo acepta peticiones del origen definido en FRONTEND_URL. Puede contener varios orígenes separados por coma.

Roles: requireRole('admin') protege todos los endpoints de administración. Las páginas del frontend también comprueban el token y el rol al cargarse.

XSS: todo el contenido de usuario insertado en el DOM pasa por escapeHtml() antes de renderizarse.

11. Funcionalidades principales

Feed: mezcla posts oficiales (solo el admin puede publicar) y posts de alumnos y profesores, ordenados por fecha. Los posts admiten texto e imagen. Las noticias oficiales se marcan con la etiqueta "Noticia del centro".

Likes y comentarios: el botón de like hace una animación al pulsarlo. Los comentarios se abren en un pop-up. El autor de un comentario y el admin pueden borrarlo desde el menú de tres puntos.

Perfiles: página de perfil con estadísticas en tarjetas (posts, seguidores, siguiendo), bio, rol y lista de publicaciones. El avatar muestra un indicador de actividad: punto verde si el usuario estuvo activo en la última hora, naranja en las últimas 24 horas.

Seguimiento: aviso de confirmación y opción de deshacer.

Mensajería: conversaciones directas y grupos con nombre. El chat se actualiza automáticamente cada 3 segundos.

Notificaciones: se generan para likes, comentarios y follows. El icono muestra el número de no leídas.

Usuarios sugeridos: hasta 8 usuarios aleatorios que el usuario actual no sigue, con indicador de actividad reciente y botón de seguir directo.

Modo oscuro: disponible en todas las páginas.

12. Panel de administración

Solo accesible para el rol admin. Tiene dos secciones.

Solicitudes de registro: lista las solicitudes pendientes con nombre, usuario y email. El admin puede aprobar asignando un rol (alumno o profesor) o rechazar. Al aprobar se crea el usuario en la tabla users.

Gestión de usuarios: lista todos los usuarios activos con avatar, nombre, email, rol, fecha de registro y último login. Desde aquí se puede cambiar el rol con un desplegable que se guarda automáticamente, suspender o reactivar la cuenta, generar un PIN de recuperación que aparece en pantalla para que el admin se lo comunique al usuario, o eliminar la cuenta de forma definitiva.

La lista tiene búsqueda por nombre y filtro por rol en tiempo real, sin peticiones adicionales al servidor.

En el feed, el admin ve las solicitudes pendientes en el sidebar izquierdo y estadísticas globales (usuarios activos, publicaciones totales y solicitudes pendientes) en el sidebar derecho.

13. Recuperación de contraseña

El sistema usa pins de 6 dígitos con expiración de 15 minutos, almacenados en la tabla password_resets.

Funciona de la siguiente manera: el usuario le indica al administrador que necesita recuperar su contraseña. El admin genera un pin desde el panel de administración, donde aparece en pantalla. El admin se lo comunica al usuario por el medio que tenga a mano, y el usuario lo introduce junto con su nueva contraseña en la pantalla de recuperación de index.html.

Este diseño es posible por el contexto de un centro educativo, donde el administrador tiene contacto directo con los usuarios y no se depende de ningún servicio de correo externo para que funcione.